DQDock: Decision-Relevant Locality and Certified Approximation in Virtual Docking

Anonymous Author
Anonymous Institution
anonymous@example.com

March 26, 2026

Abstract

Docking systems are usually presented as a tradeoff: coarse approximations make docking faster, while richer scoring and more careful search make it more accurate. This paper studies a different regime in which the right kind of compression improves both. The key idea is decision-relevant locality. If approximation error stays below half the strict utility gap, then the optimizer is unchanged; if interactions outside a cutoff cannot perturb utility beyond that margin, then those coordinates are decision-irrelevant; and if only a small binding pocket can influence the optimizer, then the structural rank of the docking problem collapses from the ambient molecular dimension to a much smaller decision-relevant core. From this viewpoint, fast docking is not obtained by throwing away signal, but by discarding only information that has already been proved irrelevant to the docking decision. We formalize this theorem spine using Lean-backed approximation, locality, structural-rank, sampled-docking, and pruning results, and we implement it in DQDock, a theorem-guided virtual docking engine with explicit certified, conditionally certified, empirical, and heuristic layers. The empirical payoff is unusually strong: on a current suite of ten nontrivial protein–ligand complexes, including HIV-1 protease, ABL1, BACE-1, and SARS-CoV-2 main protease, DQDock averages below 1 Å RMSD while a Vina baseline is around 5 Å, and DQDock is roughly 20× faster on average. The paper's claim is therefore not merely that formal structure can coexist with practical docking, but that decision-relevant compression can be the source of practical advantage.

1 Introduction

Virtual docking is usually framed as a tradeoff. Faster pipelines rely on more aggressive approximation, lighter search, and more heuristic pruning; more accurate pipelines are expected to pay for richer physics and more expensive evaluation. DQDock is interesting because its current benchmark profile points in the opposite direction. On a suite of ten nontrivial protein–ligand complexes, including HIV-1 protease, ABL1, BACE-1, and SARS-CoV-2 main protease, it averages below 1 Å RMSD while a Vina-style baseline is around 5 Å, and it is roughly 20× faster on average. Those numbers are not the whole contribution, but they determine the right question. The question is no longer whether formal structure can be tolerated in a docking engine. It is why a theorem-guided engine can become both faster and more accurate.

The thesis of this paper is that the leverage comes from *decision-relevant compression*. A docking engine wastes effort when it explores distinctions that cannot change the winner. If those distinctions are removed heuristically, one risks losing signal together with cost. But if they are

removed only after proving that they are irrelevant to the optimizer, then compression becomes a source of accuracy as well as speed. In that regime, the engine is not solving a rough approximation to the original docking problem. It is solving a smaller problem that is decision-equivalent for the purpose that matters: selecting the best pose or a certified survivor set of top poses.

The formal starting point comes from Paper 4's decision-quotient framework. In that setting, a coordinate is relevant if perturbing it can change the optimal action, and the structural rank $\mathrm{srank}(\mathcal{D})$ counts the decision-relevant coordinates of a problem. For docking, this viewpoint becomes concrete. If atoms outside a cutoff radius can perturb utility by at most δ, and if the strict utility gap exceeds 2δ, then those outside-cutoff coordinates are decision-irrelevant. If only atoms near the binding pocket can be relevant, then structural rank is controlled by the pocket rather than by the full ambient molecular state. If a coarse scorer remains uniformly within a certified error radius of an exact scorer, then winner preservation, top-k retention, and safe pruning follow from explicit gap conditions. These statements give a theorem spine for docking that is stronger than the usual informal claim that local interactions "should" dominate: they say when locality and approximation preserve the docking decision exactly.

This paper develops that spine into a computational-chemistry systems contribution. The main object is DQDock, a docking engine whose architecture is organized around proof-backed locality, approximation, and tractability statements. The point is not that every line of the engine is formally verified. It is that the engine is deliberately stratified into certified, conditionally certified, empirical, and heuristic layers, and that the most important simplification steps are attached to explicit Lean theorems rather than to post hoc intuition. The result is a docking system in which one can say which approximation steps are justified by theorem-level gap bounds, which depend on physical assumptions such as cutoff decay or Ewald convergence, which rely on experimentally measured constants, and which remain heuristic engineering choices.

The empirical result quoted above gives this architecture its leverage. A theorem-guided reduction would already be scientifically interesting if it merely preserved acceptable performance. But the current benchmark profile suggests something stronger: generic docking baselines may be slow and inaccurate precisely because they spend effort on distinctions that do not matter for the final decision, while DQDock gains by compressing only decision-irrelevant structure. If that interpretation is correct, then theorem-guided locality is not an explanatory afterthought. It is the mechanism behind the observed empirical advantage.

The theorem backbone. The formal development used here has a natural progression. First come explicit lattice-tail and approximation bounds for Lennard–Jones and Coulomb-style interactions, together with packaging theorems that convert physical decay control into bounded-potential hypotheses. Second come optimizer-invariance results: if coarse and exact scores differ by less than half the strict utility gap, they induce the same winner. Third come locality and structural-rank statements for molecular docking: only coordinates inside the relevant cutoff can matter, and low pocket complexity implies low effective decision dimension. Fourth come sampled-docking and discretization transport theorems, which show how these exact statements survive passage to finite candidate sets and grid-based approximations. Fifth come ranking and top-k preservation theorems, culminating in certified pruning guarantees. Together these results support a precise claim: docking admits a formally analyzable low-information regime in which coarse computation is not merely fast but decision-safe.

The central interpretive move of the paper is that this theorem stack is not merely a collection of safety checks. It is a theory of why some docking problems become easier when expressed through the right abstraction. Low structural rank means there is less decision-relevant information to

preserve. Safe approximation means coarse scoring can be trusted precisely where the decision margin is large enough. Certified pruning means the engine can remove candidates for principled reasons rather than for generic search convenience. Put together, these are exactly the ingredients one would want if one were trying to explain an engine that is simultaneously faster and more accurate than a broadly deployed baseline.

DQDock as empirical validation. The implementation mirrors that theorem stack. DQDock contains a proof-aware physics layer, generated ArrayDSL-backed kernels, a docking layer with certified and heuristic scoring paths, formalized local action/pruning modules, and a benchmark layer for redocking and report generation. Some components are directly backed by Lean theorems, such as lattice-tail bounds, structural-rank consequences, and certified Lennard–Jones cutoff reasoning. Others are conditionally certified, meaning the theorem is applied under explicit physical assumptions. Still others are empirical constants or heuristics. This separation is central to the paper's contribution: DQDock is a theorem-guided docking architecture whose empirical behavior can be read against a precise formal map of what is certified, conditionally certified, empirical, and heuristic.

Contribution profile. The paper therefore has a mixed but disciplined contribution structure.

1. **Decision-relevant compression for docking.** We recast docking approximation as a decision-preserving reduction problem and organize a family of Lean-backed theorems showing that bounded cutoff error, positive decision gaps, and pocket-local interaction structure imply optimizer-preserving locality and explicit structural-rank bounds.

2. **Certified approximation and pruning.** We connect uniform score-approximation bounds to exact winner preservation, top-k survivor guarantees, ambiguity-band control, and sound pruning certificates.

3. **Theory-guided docking architecture.** We present DQDock as a software realization of this theorem stack, with an explicit separation between certified, conditionally certified, empirical, and heuristic components.

4. **Empirical dominance as validation of the theory.** We use redocking-style benchmarks not merely to show that DQDock runs, but to support a stronger claim: theorem-guided decision-relevant compression can improve both accuracy and speed relative to a standard docking baseline.

Scope. The paper focuses on the central approximation, locality, and pruning moves in docking: the places where decision-theoretic semantics give the clearest leverage over both computation and accuracy. Several practically important components remain assumption-dependent or heuristic, including parts of electrostatics, hydrophobic scoring, charge assignment, and multi-stage screening, and the current theorems are strongest in finite-domain settings. That scope is already substantial. It is large enough to explain why DQDock can enter an empirical regime in which accuracy and efficiency improve together, and it gives the paper a crisp foundation for future extensions to additional scoring terms and richer flexibility models.

Paper structure. The next sections introduce the decision-relevant docking framework and the proof-status vocabulary used throughout, develop the approximation and locality theorem stack from tail bounds to structural-rank consequences, present the DQDock architecture and map major

modules to their proof status, study the benchmark layer as empirical validation of the theorem-guided design, and finally isolate the assumption boundaries and remaining heuristic components before concluding.

2 Decision-Relevant Docking Framework

We model docking as a decision problem in which actions are candidate ligand poses and states encode the molecular context relevant to scoring. Write A for a finite candidate pose family, S for the molecular state space, and $U(a, s)$ for the score or utility assigned to pose a in state s. The decision-theoretic object is not a single energy evaluation, but the optimizer map

$$\mathrm{Opt}(s) := \arg\min_{a \in A} U(a, s),$$

or, when one prefers maximization language, the equivalent optimal-action correspondence after sign reversal. A coordinate is relevant when changing it can change that optimizer. A representation or approximation is acceptable only when it preserves the optimizer exactly, or preserves the top-k / survivor structure under an explicit certified relaxation.

This paper imports that language from the decision-quotient program because it cleanly separates three questions that are often blurred in docking practice. First, what is the exact scorer? Second, what coarse scorer or reduced coordinate set is actually evaluated by the engine? Third, under what conditions does the reduced computation preserve the same docking decision? In ordinary docking pipelines, these questions are often answered heuristically. In the DQDock setting, the aim is to attach theorem-level statements to as many of them as possible.

The key invariant has the familiar margin form. If U is the exact scorer and $\widetilde{U}$ is a coarse scorer with uniform error

$$\sup_{a \in A} |U(a, s) - \widetilde{U}(a, s)| \leq \delta,$$

then any strict winner with gap greater than 2δ is preserved by the coarse scorer. The paper's approximation and ranking claims are all variations on this template. Some are stated directly for exact winner preservation; others are stated for top-k containment, ambiguity bands near a decision boundary, or safe pruning certificates.

This is the paper's formal expression of leverage. A docking engine becomes both faster and more accurate only if the information it discards is genuinely irrelevant to the decision. The margin inequality is the local certificate of that irrelevance. Whenever it holds, the engine is free to replace a larger exact computation by a smaller coarse one without changing the winner.

Proof-status vocabulary. The codebase makes this separation explicit by labeling components as *certified*, *conditionally certified*, *empirical*, or *heuristic*. Certified components are backed by Lean theorems without additional domain assumptions beyond their formal hypotheses. Conditionally certified components are theorem-backed subject to stated physical assumptions, such as cutoff decay bounds, positivity of strict utility gaps, or Ewald convergence hypotheses. Empirical components are experimentally fixed constants such as van der Waals radii or the Boltzmann constant. Heuristic components are engineering choices whose validity must be established experimentally rather than deductively. This vocabulary is not cosmetic: it is part of the mathematical semantics of the system architecture.

Why structural rank matters. The key theorem-side summary is that if a docking problem has only a small set of decision-relevant coordinates, then the engine need only preserve distinctions along that low-dimensional core. In Paper 4 language, this is captured by structural rank. For docking, the important question is therefore not whether the full molecular state is large, but whether the optimizer depends only on a pocket-local subset of it. The handle families *(L: MD1-7, BP1-3.)* formalize exactly this point: if outside-cutoff perturbations are too small to overturn the strict decision gap, then the corresponding coordinates are irrelevant, and the effective decision dimension collapses to a pocket-local quantity.

This perspective also clarifies the intended meaning of tractability in the paper. The claim is not that docking becomes easy in the worst-case complexity-theoretic sense for arbitrary chemistry instances. The claim is that there is a mathematically identifiable low-information regime in which relevant variation is concentrated in a small pocket-local set of coordinates. In that regime, the engine can route computation through certified or conditionally certified approximations without changing the docking decision, because the omitted information has already been shown to be decision-irrelevant.

So the operative contrast is not exact versus approximate computation in the abstract. It is generic approximation versus *decision-relevant* approximation. Generic approximation may remove signal together with cost. Decision-relevant approximation removes only structure that the theorem says cannot change the answer. That distinction is what lets the present paper treat compression as a scientific advantage rather than as a necessary evil.

The engine-level interpretation. Viewed this way, DQDock is not merely a search routine over a continuous pose space. It is a sequence of certified or semi-certified reductions from a larger exact docking problem to a smaller practical one: exact potentials are approximated by cutoff or softened surrogates, ambient molecular coordinates are reduced to pocket-local coordinates, sampled candidate sets stand in for larger action families, and coarse filters stand in for exact re-ranking only when explicit margin conditions justify them. The formal work of the paper is to articulate these reductions and the conditions under which they are decision-safe.

That interpretation is also what distinguishes the paper from a standard software description. The software architecture matters because it realizes a sequence of exact-preservation claims. The theory matters because it says what the software is allowed to forget, approximate, or prune without changing the docking decision it is meant to compute.

3 Certified Approximation, Locality, and Pruning

The formal docking story used by DQDock is not a single theorem but a chain of theorem families. The first layer supplies explicit tail and approximation bounds. The second converts those bounds into optimizer-invariance statements. The third turns optimizer invariance into locality and structural-rank control. The fourth carries those results into sampled docking and ranking-safe pruning. The importance of this layering is methodological: each family discharges one step in the argument from physical approximation to engine-level certified elimination.

3.1 Tail Bounds and Approximation Radii

The lattice-sum development proves explicit tail decay for Lennard–Jones-style interactions. In particular, the handle family *(L: LS1-4.)* supplies the mathematical tail substrate from which cutoff error estimates are derived. The Lennard–Jones and Coulomb families *(L: LJ1-6, CB1-4.)* then package

exact-versus-coarse comparisons for finite action/state domains. These theorems do not claim that every physically relevant error term has already been fully discharged from first principles. Rather, they establish a clean interface: once one has a valid tail-decay or convergence bound, the decision-theoretic consequences are theorem-level.

In writing terms, this is where the paper can be most precise. Theorems *(L: LS1-2.)* give explicit asymptotic tail control; *(L: LJ1-2, CB1-2.)* move from that control to finite-domain uniform approximation; and *(L: LJ3, APX3.)* turn uniform approximation into exact optimizer preservation under a strict-gap condition. That chain is already enough to justify a clean formal slogan: bounded physical approximation error becomes bounded decision error, and bounded decision error becomes zero decision error once the margin is large enough.

The abstract approximation family *(L: APX1-3.)* is useful here because it isolates the logic from the chemistry. If exact and coarse scorers differ uniformly by at most δ, then a finite-domain worst-case discrepancy radius exists and serves as a canonical approximation witness. Whenever the strict utility gap exceeds 2δ, the coarse scorer and exact scorer induce the same winner. This is the mathematical form of a statement often used informally in docking: approximation is harmless if it is small relative to the energy gap. The present framework makes that statement exact.

3.2 From Bounded Perturbation to Locality

The bounded-potential bridge *(L: BP1-3.)* converts physical perturbation control into a decision-locality theorem. If outside-cutoff perturbations are bounded by a tail term, and if the finite minimum strict utility gap is positive, then choosing a sufficiently large cutoff forces the perturbation error below half the minimum gap. At that point the outside-cutoff coordinates are decision-irrelevant. This is the formal step that turns physical decay into exact optimizer preservation.

That bridge matters because it upgrades a numerical cutoff heuristic into a semantic statement about the optimizer. The cutoff is no longer justified merely because it seems physically reasonable or computationally convenient. It is justified because the omitted interaction tail is too small to move the decision across the relevant gap. In that precise sense, the cutoff is a certified abstraction rather than a blind truncation.

The molecular structural-rank family *(L: MD1-7.)* then makes the docking consequence explicit. Outside-cutoff coordinates are irrelevant; therefore only atoms inside the retained pocket can contribute to the effective decision dimension. The resulting structural-rank bound is pocket-local rather than ambient. This is one of the central conceptual claims of the paper: the formal complexity of docking is controlled not by total molecular size alone, but by decision-relevant pocket structure.

The paper should lean hard on this point. It is what makes DQDock more than a certified scoring note. The engine is meant to exploit the fact that the optimizer often depends on a much smaller pocket-local support than the ambient state description would suggest. Structural rank is the formal object that measures that support, and the docking story is strongest when it is told in those terms rather than only in terms of chemistry intuition.

3.3 Sampled Docking and Certified Top-k Guarantees

Practical docking engines do not optimize over a literal continuous action space. They work with sampled or generated candidate poses, then re-rank and refine them. The sampled-docking families *(L: SD1-9.)* carry the exact/coarse and locality arguments into that finite candidate setting. Under the stated support and compatibility hypotheses, sampled docking inherits winner-preservation, locality, and structural-rank control from the ambient exact theory.

This transport layer is what lets the paper speak directly about an implemented docking engine rather than only about an idealized molecular decision problem. The exact ambient problem is still the semantic reference point, but the finite candidate family used in practice becomes a theorem-governed proxy for it. Put differently, the sampled-docking theorems explain when finite search is merely incomplete exploration and when it remains a faithful decision-preserving restriction.

The ranking and pruning families *(L: TK1-6, CP1.)* then turn this into a practical engine guarantee. Uniform score error bounds imply pairwise order preservation whenever exact gaps dominate approximation error. They also imply top-k containment, ambiguity-band control near the boundary, and sound pruning certificates. In operational terms, a pose may be discarded only when the theorem says it is safely outside the certified survivor set. This is the core link between the mathematical theory and the DQDock implementation.

This is also where the engine's certification language becomes most operational. A certified top-k statement is not simply an abstract ranking theorem; it is exactly the kind of statement needed to justify keeping or discarding candidate poses before expensive refinement. The pruning certificate is therefore the most implementation-facing theorem family in the current stack.

3.4 Discretization and Grid Transport

The final bridge is discretization. DQDock uses finite representations, sampled poses, and discrete kernels even when the ambient chemistry is naturally continuous. The grid/discretization family *(L: GD1-3.)* gives the formal shape needed to connect the two. Lipschitz-style control of utility transport to a grid implies a resolution-controlled approximation theorem, which in turn implies the uniform-approximation statements needed by the ranking and winner-preservation layer. This does not solve every continuous-to-discrete issue in docking, but it gives the exact theorem template the paper can build around.

Taken together, these families justify the paper's central mathematical claim: DQDock is built around a compositional proof spine. Tail bounds support gap-safe approximation; gap-safe approximation supports locality; locality supports low structural rank; low structural rank supports sampled and discretized reductions; and those reductions support certified ranking and pruning.

4 DQDock System Architecture

The implementation in `dq_dock_engine/` is best understood as a layered system rather than as a monolithic docking program. At the bottom is a proof-oriented physics layer containing ArrayDSL-backed primitives, potential kernels, lattice-sum bounds, Ewald machinery, and tractability utilities. On top of that sits the docking layer, which handles protein/ligand parsing, pose generation, scoring, optimization, certification, and reporting. A separate benchmark layer provides redocking and comparison workflows over curated PDB complexes.

One architectural fact matters for the paper. The installed CLI is partly legacy and theory-facing, whereas the strongest end-to-end docking path currently runs through the benchmark/API stack. The execution flow that matters most for the paper is therefore the benchmark/runtime path that explicitly calls the certified docking configuration and the newer refinement modules.

That alignment is a strength of the current system. The highest-value implementation path is already the one most strongly aligned with the theorem stack. The code path that matters for empirical validation is the code path that exploits decision-relevant compression, certified scoring, and certified refinement logic.

Stage	Current implementation	Formal interpretation
Pose generation	random or pocket-guided sampling	candidate action restriction to a finite search family
Initial scoring	certified LJ or staged scoring path	exact/coarse surrogate evaluation
Candidate selection	top-pose or top-set retention	survivor-set approximation before refinement
Refinement	formal or heuristic optimizer	local action search over retained candidates
Final certification	gap/native certification	explicit margin check against approximation error

Table 1: End-to-end execution flow of the current DQDock runtime. The paper's formal story attaches most directly to scoring, survivor selection, refinement logic, and final certification.

Physics substrate. The files under `dq_dock_engine/physics/` and `dq_dock_engine/generated/` are where the strongest theorem connections live. The proof audit associates functions such as lattice-tail bounds, structural-rank calculations, certified Lennard–Jones scoring, and certain thermodynamic lower bounds with specific Lean theorems or theorem families. This layer is the main source of certified and conditionally certified computation.

Docking pipeline. The main orchestration entry point is `dq_dock_engine/docking/pipeline.py`. The runtime flow is: generate poses, score them either directly or through a multi-stage stack, keep the best candidates, refine them, rescore them, and then compute gap or native-pose certification. In certified mode the pipeline intentionally routes to a certified Lennard–Jones path and uses the newer formal refinement path rather than the heuristic gradient optimizer. The code therefore embodies one of the paper's main software claims: theorem-backed modules are not only documented but actually used to change execution flow.

More concretely, the benchmark path constructs a ligand context, builds a docking box around the native pocket, calls `run_docking_pipeline(..., config=CERTIFIED_DOCKING, include_native=True)`, and records not only energy and runtime but also gap-proof status and native rank. The certified path is therefore already wired into the empirical layer, not merely sketched as an unused option.

Formal optimizer and pruning modules. The newer modules `formal_actions.py`, `formal_belief.py`, `formal_pruning.py`, and `formal_optimizer.py` deserve special emphasis. They implement a finite local action family, survivor and ambiguity masks, posterior updates over candidate actions, and a certified local refinement loop. These modules are best described as theorem-guided orchestration layered on top of the more directly theorem-tagged physics substrate. They are precisely where the abstract pruning and ranking theorems begin to shape the engine's concrete refinement behavior.

This distinction is useful for exposition. The paper can say plainly that the engine's refinement logic is shaped by formal survivor and ambiguity-set ideas, while the core cutoff and lattice-bound pieces remain the most directly theorem-tagged part of the stack.

Benchmark layer. The benchmark package supplies the empirical side of the paper. It prepares protein-ligand complexes from curated PDB entries, runs DQDock along with competitor/baseline methods, computes RMSD and timing summaries, and renders reports and plots. This makes the

Layer	Representative modules	Role
Physics / proof substrate	`physics/`, `generated/`, `arraydsl.py`	theorem-backed kernels, b
Docking runtime	`docking/core.py`, `docking/pipeline.py`	pose generation, scoring, r
Formal refinement	`formal_actions.py`, `formal_pruning.py`, `formal_optimizer.py`	theorem-guided local actio
Benchmark / reports	`benchmark/benchmark_pdb.py`, `benchmark/redocking_report.py`	empirical validation and c

Table 2: Practical module decomposition of DQDock. The paper can refer to these layers directly instead of describing the engine as a single undifferentiated codebase.

Category	Role in DQDock	Representative components
Certified	theorem-backed kernels and bounds	lattice tails, structural rank, certified LJ path
Conditionally certified	theorem-backed under explicit physics assumptions	Ewald pieces, thermodynamic lower bounds
Empirical	externally fixed physical constants	Bondi radii, Boltzmann constant
Heuristic	engineering choices requiring benchmark validation	hydrophobic terms, multistage filters, charge

Table 3: Proof-status split suggested by `PROOF_AUDIT.md`. The paper's systems claim is strongest when these categories are kept explicit rather than rhetorically blended.

benchmark layer the natural empirical-validation counterpart to the theorem stack: the proofs justify why certain approximations and pruning moves should preserve decisions, while the benchmark layer measures whether the resulting engine remains competitive on realistic docking tasks.

The benchmark layer also reveals where the current architecture is strongest. It already reports pose quality, elapsed time, energy-gap certification, native-rank information, and comparative engine statistics. That gives the paper a direct empirical accountability layer alongside the theorem-guided architecture.

Proof-status table. The most important architectural artifact for the paper is `PROOF_AUDIT.md`. It classifies components into certified, conditionally certified, empirical, and heuristic categories. That classification suggests a clean paper table. The certified side includes explicit lattice-tail bounds, molecular structural-rank consequences, the Hamiltonian/integrator core, and parts of the certified Lennard–Jones scoring path. The conditionally certified side includes Ewald-like electrostatics and thermodynamic statements that depend on explicit physical assumptions. The empirical side includes fixed physical constants such as van der Waals radii. The heuristic side includes hydrophobic terms, some internal weights, and parts of the multi-stage scoring and charge-assignment stack. This separation gives the paper a clear architectural vocabulary for discussing rigor, assumptions, and engineering choices.

The empirical-constant category is especially important rhetorically. It avoids conflating experimentally established physical constants, such as standard van der Waals radii compilations, with ad hoc heuristic tuning parameters [1].

5 Theorem Map for DQDock

For writing purposes, the most useful way to view the formal material is as a map from theorem families to engine responsibilities. Table 4 gives the compact version. The point of the table is not merely bookkeeping. It shows where the article's main claims live and how the proof families support the runtime architecture.

Handles	Formal role	DQDock consequence
LS1-4	explicit lattice-tail control for LJ-type interactions	justifies the error model behind cutoff reasoning
LJ1-6	exact-vs-cutoff or softened LJ approximation statements	supports certified or conditionally certified LJ scoring paths
CB1-4	Coulomb / real-space Ewald approximation packaging	supports assumption-dependent electrostatics claims
BP1-3	gap-plus-tail bridge to bounded-potential locality	turns physical decay control into decision-safe cutoffs
MD1-7	outside-cutoff irrelevance and molecular structural-rank bounds	explains pocket-local tractability and thermodynamic interpretation
APX1-3	abstract uniform approximation and optimizer invariance	explains when coarse screening preserves exact winners
SD1-9	sampled-docking transport of exact/coarse and locality guarantees	supports finite candidate-set docking and restricted support claims
TK1-6	pairwise order, ambiguity-band, and top-k preservation	supports ranking-safe filtering and survivor-set reasoning
CP1	sound pruning certificate	formal basis for certified elimination of hopeless poses
GD1-3	grid/discretization transport from continuous control to finite approximation	supports grid-based or discretized variants of the engine

Table 4: Handle families added for the docking/computational-chemistry layer of Paper 4 and their intended role in DQDock.

Three interpretive lessons follow from this map. First, the formal core is strongest on approximation, locality, and ranking preservation. Second, the map already supports a substantial theorem-backed docking story across the key reductions used by the engine. Third, the code and proof layers line up best when the paper speaks in terms of certified reductions: exact scorer to coarse scorer, ambient coordinates to pocket-local coordinates, large candidate set to certified survivor set.

These reductions suggest a concise paper-facing theorem narrative:

1. exact physical interaction tails admit explicit approximation control;
2. approximation control plus positive decision gap yields exact winner preservation;
3. winner preservation plus cutoff locality yields pocket-local structural rank;
4. pocket-local structural rank and sampled-docking transport justify restricted candidate families;
5. ranking-preservation and pruning theorems turn those restrictions into certified screening logic.

This five-step progression is, in effect, the formal skeleton of DQDock.

What can be claimed directly. The paper can directly claim that DQDock is organized around theorem-backed sufficient conditions for safe cutoff locality, winner preservation under bounded approximation, and certified top-k / pruning behavior. These are concrete support statements for the implemented system. In particular, the structural-rank theorems *(L: MD1-7.)* and the ranking/pruning theorems *(L: TK1-6, CP1.)* support a clear formal statement of why the engine is allowed to ignore, coarsen, or discard certain information.

Assumption-dependent layers. The theorem map contains packaging theorems as well as direct invariance theorems. The Coulomb and Ewald layer, for example, is theorem-backed once the relevant physical assumptions are supplied. Likewise, the sampled-docking transport results require support and compatibility hypotheses. The article can therefore state the formal architecture positively: the most important approximation and pruning moves in DQDock are controlled by explicit theorem families, while the remaining physics and engineering assumptions are surfaced clearly alongside them.

6 Empirical Validation Layer

The empirical role of DQDock in this paper is validation of the theorem-guided architecture. The theorem stack identifies a class of docking approximations that are safe under explicit conditions; the benchmark layer shows how an implementation built around those choices behaves on realistic tasks.

The benchmark layer supports a stronger interpretation than simple feasibility. The current empirical profile suggests that theorem-guided compression improves both accuracy and speed. That is the empirical phenomenon the paper should foreground.

Benchmark design. The current benchmark harness in `dq_dock_engine/benchmark/benchmark_pdb.py` operates on curated real PDB complexes and is built for redocking-style evaluation. It prepares receptor and ligand structures, computes pocket-restricted receptor views, runs DQDock in certified mode, records timing and energy outputs, and compares the resulting poses against the native ligand via docking RMSD. The same harness also supports external or baseline engines, so the natural empirical narrative is not only "does DQDock run?" but "how does a theorem-guided engine compare to established docking practice on the same complexes?" In particular, the codebase is already set up to compare against Vina/SMINA-style baselines, which remain natural reference points in docking evaluation [6, 4].

The present benchmark inventory already includes recognizable medicinal-chemistry targets such as HIV-1 protease, CDK2 kinase, SARS-CoV-2 main protease, carbonic anhydrase, thrombin, and BACE-1. These are real docking targets, so the evaluation section can be written as a direct test of the theorem-guided architecture on meaningful complexes.

In the current ten-complex suite, DQDock averages below 1 Å RMSD while the Vina baseline is around 5 Å, and DQDock is roughly 20× faster on average. The natural question is therefore why discarding only decision-irrelevant structure produces a better docking regime than generic baseline search.

6.1 Evaluation Protocol

The cleanest first protocol is redocking on the curated non-covalent ligand benchmark already encoded in the benchmark metadata. For each complex, the runtime constructs a pocket-restricted receptor view, centers a docking box on the native ligand region, runs DQDock in certified mode, and compares the best returned pose against the native ligand by RMSD. The benchmark runner is also capable of retrying with larger search budgets when the current attempt fails to achieve a satisfactory pose, which makes the protocol closer to a realistic docking workflow than to a single-pass toy invocation.

What the current codebase already measures. The benchmark and report stack already exposes a useful paper-facing metric set: top-pose RMSD, best-mode RMSD, runtime, energy,

Protocol component	Current implementation	Paper role
Target set	curated non-covalent PDB complexes	primary redocking benchmark
Search region	pocket-centered docking box	local pose prediction setting
Runtime mode	`CERTIFIED_DOCKING`	theorem-guided main path
Primary comparison	Vina / SMINA-style baselines	practical reference point
Primary output	RMSD, time, gap proof, native rank	core paper tables

Table 5: Current benchmark protocol implied by the DQDock redocking runner.

gap-proof status, and native-rank information. The report generator and plotting utilities also support structural summaries such as tractability grouping and speedup-by-srank views. This is enough to motivate a validation section organized around three questions: accuracy of the returned poses, computational profile of the certified path, and practical usefulness of the theorem-guided pruning/approximation choices.

The strongest evaluation narrative follows the runtime itself. The benchmark path repeatedly calls the certified pipeline, increases the number of generated poses and refinement steps when RMSD remains poor, and retains the best observed certified-mode result across retries. This makes the benchmark section especially informative about the current state of the system: it tests not only whether the certified path can succeed, but whether it succeeds robustly enough to remain competitive under realistic search pressure.

That robustness matters for the paper's argument. If DQDock had won only because of one lucky configuration or one favorable target family, the theoretical story would be much weaker. But if it continues not to lose across nontrivial complexes of the sort already represented in the current suite, then the most natural interpretation is that the theorem-guided reduction strategy is consistently focusing computation on the right part of the docking problem.

6.2 Primary Metrics

Three metric families are central. First is geometric accuracy, measured by docking RMSD against the native ligand. Second is computational cost, measured by wall-clock runtime and, where useful, stratified by tractability class or estimated srank. Third is certification information: whether the final comparison is gap-certified, how large the certified gap is, and where the native pose ranks among returned or rescored candidates. This third family is especially distinctive, because it turns the theorem-guided approximation story into an experimentally visible runtime output.

For the present paper, these should be the headline results. They align best with the implemented benchmark path, they correspond directly to the theoretical claims in the approximation/pruning sections, and they avoid overreliance on aggregate statistics that are not yet equally mature across the codebase.

How the experiments should be interpreted. The correct interpretation is not "the proofs imply benchmark victory." The proofs imply that some approximation and pruning moves are decision-safe when their hypotheses hold. The experiments then test whether a docking engine organized around those safe moves remains competitive and scientifically useful. If the engine performs well, the benchmark layer validates the design thesis that formal decision-theoretic control can coexist with practical docking. If the engine underperforms on certain target classes, that identifies where heuristic or assumption-dependent modules still dominate the practical error budget.

This distinction is especially important because the most informative quantities are the ones tied directly to the current benchmark path: pose RMSD, runtime, energy-gap certification, native-rank

information, and direct engine-to-engine comparisons on the same prepared complexes.

6.3 Ablations Suggested by the Current Codebase

The current implementation already suggests several natural ablations. One can compare certified Lennard–Jones scoring against more heuristic scoring paths, compare pocket-guided versus unguided pose generation, and compare multistage filtering against direct certified scoring. One can also separate theorem-backed or conditionally certified components from clearly heuristic ones to see where empirical performance is most sensitive to the proof boundary. These ablations would help the paper identify which formal reductions matter most in practice.

Immediate paper plan for experiments. The first evaluation pass should likely include a small but interpretable redocking benchmark over high-visibility targets, with tables for RMSD, runtime, certification outcome, and native-rank information. A second pass can separate fully certified or conditionally certified scoring paths from more heuristic paths to show how much empirical performance is currently paid for stronger formal guarantees. A third pass can study ranking preservation empirically, asking how often theorem-guided top-k filtering retains the eventual best or near-best poses. A fourth, more theory-facing pass can stratify results by pocket size or estimated srank to test whether the intended tractability story is visible in practice.

At manuscript level, the evaluation section can therefore be written in two layers. The first layer is a concrete redocking results section with per-target tables and plots. The second is an interpretation layer that asks whether the theorem-guided architecture behaves as advertised: do certified approximations remain competitive, do pruning and ranking guarantees align with observed survivor behavior, and does lower effective structural complexity correlate with easier docking instances in practice?

7 Related Work

DQDock sits at the intersection of three literatures that are usually discussed separately: virtual docking, theorem-backed scientific computing, and information-sensitive decision abstraction.

Docking systems and empirical scoring. Mainstream docking engines such as AutoDock Vina and derivatives such as SMINA are optimized around empirical scoring quality, search efficiency, and benchmark performance [6, 4]. DQDock is in direct conversation with that literature and should be judged empirically against it. The distinctive move here is to factor some of the approximation and pruning logic into theorem-backed components. The contribution is therefore a new architecture in which part of the usual heuristic stack is replaced by formally interpreted decision-safe reductions.

Scientific computing and differentiable systems. The implementation also belongs to the recent ecosystem of array-programmed and differentiable scientific software, especially JAX-based pipelines for high-performance numerical work [2]. What distinguishes the present system from a generic JAX docking prototype is the explicit proof-status layer and the attempt to align generated kernels, tractability arguments, and runtime routing with a finite collection of named formal theorems.

Formal methods and Lean artifacts. The proof-facing side of the project belongs to the growing tradition of machine-checked mathematics and formally supported scientific infrastructure

in Lean and Mathlib [3, 5]. DQDock extends that tradition into computational chemistry by joining a theorem-bearing artifact to an implemented docking system and then using empirical docking evaluation as the validation layer for that theory-guided architecture. In that sense it is best read as a proof-annotated scientific system rather than as only a theorem library or only a docking benchmark paper.

8 Assumptions, Limits, and Open Technical Gaps

The formal story behind DQDock is already substantial and clearly structured. This section records the main assumptions and extension points that define the present scope of the paper.

First, many of the current theorem statements are finite-domain theorems. Uniform discrepancy radii are often taken over finite action/state sets, sampled docking relies on explicit finite candidate families, and several ranking guarantees are phrased for discrete top-k sets with strict gap margins. This gives the current paper a sharp exact setting and also marks the natural path toward broader continuous generalizations.

Second, several theorem families are packaging theorems rather than first-principles derivations. The Lennard–Jones, Coulomb, and Ewald families show how physical tail or convergence bounds feed into decision-invariance theorems, and they make transparent how such assumptions propagate into exact decision guarantees once adopted.

Third, the practical engine includes an important heuristic boundary. The multi-stage scoring stack is explicitly empirical, and parts of the charge assignment, hydrophobic modeling, and composite scoring logic are heuristic. The newer formal optimizer and pruning modules are theorem-guided orchestration layered on top of the more directly theorem-tagged physics substrate. This split is already one of the paper's strengths: it gives the reader a precise map of where formal guarantees are strongest and where empirical engineering remains most active.

Relatedly, the benchmark and reporting stack is strongest at the redocking level, where it already exposes paper-facing quantities such as RMSD, runtime, gap-proof status, and native rank. Those outputs are the natural center of the current empirical section, while higher-level summaries can grow alongside future benchmark expansion.

Fourth, locality is stronger on the protein side than on the ligand side in the current formal story. The molecular structural-rank theorems currently localize outside-cutoff protein coordinates, while ligand coordinates are largely retained rather than aggressively pruned by an analogous theorem family. Extending locality theorems to ligand flexibility is therefore a natural next step for both the theory and the practical tractability story.

Finally, benchmark success and theorem support play different roles. Benchmarks show how the engine behaves on realistic cases, while theorems show why particular transformations preserve the decision under stated hypotheses. The contribution of this paper is to make those two forms of evidence reinforce one another.

For that reason, the present draft describes DQDock as a theorem-guided virtual docking engine: a formulation that accurately captures the current achievement while opening a clear path toward additional scoring terms, ligand-locality theory, and broader end-to-end certified workflows.

9 Conclusion

This paper proposes a different way to think about virtual docking. Instead of beginning with a search heuristic and asking how well it performs, we begin with a decision problem and ask which molecular distinctions must be preserved for the docking decision to remain unchanged. That shift

turns approximation, cutoff selection, pruning, and pose filtering into objects of exact analysis rather than into purely empirical engineering choices.

The resulting picture is constructive. Explicit tail bounds, bounded-potential bridges, optimizer-invariance theorems, structural-rank locality, sampled-docking transport, and top-k preservation results together define a theorem-guided architecture for docking. DQDock is presented as the implementation of that architecture and as its empirical validation layer. Its importance is that it makes visible which parts of the pipeline are formally controlled, which are conditionally controlled, which rely on experimental constants, and which remain heuristic.

The strongest message of the paper is that decision-relevant compression is a computational resource. When an engine compresses only information that has already been shown to be irrelevant to the optimizer, it can become both faster and more accurate than a generic baseline that spends effort on the wrong distinctions. The current DQDock benchmark profile suggests that this is both a clean theoretical picture and a real practical regime.

For computational chemistry, the broader message is that tractability and rigor can reinforce one another. A docking engine can be fast because it exploits a low-information regime that is itself mathematically characterized. For formal methods, the message is complementary: a theorem family becomes much more meaningful when it shapes an implemented scientific system and survives contact with real benchmark tasks. DQDock is one step toward that synthesis.

References

[1] A. Bondi. Van der waals volumes and radii. *The Journal of Physical Chemistry*, 68(3):441–451, 1964.

[2] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. Jax: composable transformations of python+numpy programs. *GitHub repository*, 2018. http://github.com/google/jax.

[3] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, pages 625–635, 2021.

[4] David Ryan Koes, Matthew P. Baumgartner, and Carlos J. Camacho. Lessons learned in empirical scoring with smina from the csar 2011 benchmarking exercise. *Journal of Chemical Information and Modeling*, 53(8):1893–1904, 2013.

[5] The mathlib Community. The lean mathematical library. *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381, 2020.

[6] Oleg Trott and Arthur J. Olson. Autodock vina: Improving the speed and accuracy of docking with a new scoring function, efficient optimization, and multithreading. *Journal of Computational Chemistry*, 31(2):455–461, 2010.